Table of contents

The Essentials

Introduction and Motivation

PCA (Principal Component Analysis) is a **linear latent decomposition** method that identifies the **underlying latent factors** structuring data.

The Fundamental Problem

Base representations of data are not always optimal. PCA seeks to find a new representation that:

- Reveals the **latent factors** (hidden) governing the process
- Uses **statistical correlation** to identify these factors
- Enables **simplification** of data through justified decimation

Key Point: PCA belongs to the family of **linear latent models**

Geometric Intuition: The Point Cloud

Imagine a 3D point cloud (like midges in a room). If you had to squash them onto a wall:

- Which wall would you choose so that the midges are **most spread out**?
- That wall is the **principal plane**
- The line in this wall where they are most aligned is the **first component**

Essential Idea: PCA finds the **skeletal structure** of the data, like a radiologist seeing bones through skin.

Maximizing Variance

Mathematical Formalization

Idea: We seek a line passing through the center where the projections are **as dispersed as possible**.

Let X be the dataset, $\{u_i\}_{i\in[\![D]\!]}$ a new **orthonormal basis** of $\mathbb{R}^D$.

The **first principal component** u_1 is chosen such that the **variance of the projected data** on u_1 is maximized.

Mathematical Details: Optimization Problem

The empirical mean and projected variance are:

$$x = \frac{1}{N}\sum_{i=1}^{N} x_i$$

$$v_{u_1} = \frac{1}{N}\sum_{i=1}^{N}(u_1^T x_i - u_1^T x)^2 = u_1^T \Sigma u_1$$

where $\Sigma = \frac{1}{N}\sum_{i=1}^{N}(x_i - x)(x_i - x)^T$ is the **covariance matrix** of the data.
The problem becomes:

$$u_1 = \arg\max_{u^T u = 1} u^T \Sigma u$$

Using **Lagrange multipliers**:

$$J(u) = u^T \Sigma u + \lambda(1 - u^T u)$$

The optimality conditions give:

$$\Sigma u_1 = \lambda_1 u_1 \quad \Rightarrow \quad v_{u_1} = \lambda_1$$

Conclusion: (u_1, λ_1) is an **eigenpair** of the covariance matrix Σ

Complete Decomposition

Continuing this process, we obtain the set of **principal components** as the eigenpairs $\{(u_i, \lambda_i)\}_{i \in [\![D]\!]}$ of Σ.

Matrix Decomposition:

$$\Lambda = U^T \Sigma U$$

where U contains the eigenvectors in columns and Λ is diagonal with the eigenvalues.

Minimizing Error

Idea: Represent each point by its projection on a line with **minimal reconstruction error** (perpendicular distance).

Variance-Error Equivalence

The variance v_{u_1} is expressed as a sum of squares:

$$v_{u_1} = \frac{1}{N}\sum_{i=1}^{N}(u_1^T(x_i - x))^2$$

> **i** Mathematical Details: Pythagorean Theorem
>
> To maximize v_{u_1}, the terms $u_1^T(x_i - x)$ must be collectively maximized.
> Since $(x_i - x)$ is fixed, this is equivalent to **minimizing the distance to the projection axis** (approximation error).
> **Alternative Formulation:**
>
> $$u_1 = \arg\min_{u^Tu=1} \sum_{i=1}^{N} \|(x_i - x) - [u^T(x_i - x)]u\|_2^2$$
>
> This formulation also gives: $\Sigma u_1 = \lambda_1 u_1$

Fundamental Equivalence:

Maximizing projection variance **Minimizing approximation error**

Interpretations

- **Regression:** A principal component is a **quadratic regression** on the data
- **Physics:** A principal component is a **subspace of minimal inertia** with respect to X

Dimensionality Reduction

Structure of the Latent Space

The latent space with basis $\{u_1, \dots, u_D\}$ has the following properties:

- **Eigenvalue Order:** By construction: $\lambda_1 > \lambda_2 > \cdots > \lambda_D$
- **Total Variance:** $\lambda = \sum_{d=1}^{D} \lambda_d = \mathrm{Tr}(\Sigma)$
- **Decorrelation:** $v_{u_d} = \lambda_d$ and $u_d^T u_{d'} = 0$ for $d \neq d'$

Latent Coordinates

The basis $\{u_1, \ldots, u_D\}$ induces **latent coordinates** y_i for the data:

$$y_i = U^T(x_i - x)$$

The PCA transformation is **linear** and consists of three steps:

- **Centering** on x
- **Rotation** using matrix U
- **Scaling** (whitening) using $\Lambda^{1/2}$

Important Limitation: PCA assumes an approximately **Gaussian distribution** and will not be relevant for non-Gaussian data (e.g., clustered data).

Low-Rank Approximation

Eckart-Young Theorem

If $\Sigma = U\Lambda U^T$ and for $K < D$, we define:

$$\Sigma_K = \sum_{d=1}^{K} \lambda_d u_d u_d^T$$

Then Σ_K is the **closest rank-K matrix** to Σ in the Frobenius norm.

> **i** Mathematical Details: Approximation Error
>
> The theorem states that:
>
> $$\arg\min_{\text{rank}(S)=K} \|\Sigma - S\|_F^2 = \Sigma_K$$
>
> where $\|\cdot\|_F$ is the **Frobenius norm**.
> The approximation error is:
>
> $$\|\Sigma - \Sigma_K\|_F^2 = \sum_{d=K+1}^{D} \lambda_d$$
>
> This error corresponds exactly to the variance lost during reduction.

Practical Truncation

Truncation of U and Λ:

$$\Sigma_K = U_K \Lambda_K U_K^T$$

Projection and Reconstruction:

- **Projection** (encoding): $\tilde{y}_i = U_K^T x_i \in \mathbb{R}^K$
- **Reconstruction** (decoding): $\tilde{x}_i = U_K U_K^T x_i + x$

Important: We go from D dimensions to K dimensions with $K < D$

Geometry and Reconstruction Quality

Relative Contribution to Variance

The contribution of each dimension:

$$c_d = \frac{\lambda_d}{\sum_k \lambda_k}$$

This metric depends on the **spectrum distribution** of eigenvalues.

Projection Ratio

The projection ratio of a data point x_i on a latent factor:

$$\rho_d(x_i) = \frac{\langle u_d, x_i \rangle^2}{\|x_i\|^2} = \cos^2(\angle(u_d, x_i))$$

The closer $\rho_d(x_i)$ is to 1, the more x_i lies on the axis u_d.

Normalization and Scaling

PCA is more effective if **all scales are similar**.

Solution: Normalization

We create the **scaling matrix** $S = \text{diag}[\sigma_1^2, \dots, \sigma_D^2]$ and use the **Mahalanobis metric**:

$$d_S^2(x, y) = (x - y)^T S^{-1}(x - y)$$

Practical Tip: Normalizing data before PCA often improves results.

Practical Application of PCA

Complete PCA Algorithm

Given dataset $X \in \Omega$:

- Compute the empirical mean $x = \frac{1}{N} \sum_{i=1}^{N} x_i$
- Center the data: $x_i \leftarrow (x_i - x)$ and form the centered matrix X
- Compute the covariance matrix: $\Sigma = \frac{1}{N} X X^T$
- Decompose into eigenvalues: $\Sigma = U \Lambda U^T$

So far: **exact** transformation

- Select the number of components K
- Define U_K, Λ_K and compute $\{\tilde{y}_i\}_{i \in [\![N]\!]}$ and/or $\{\tilde{x}_i\}_{i \in [\![N]\!]}$

Choosing the Number of Components K

Three Main Strategies:

Strategy 1: Target Dimension

$$K = d$$

where d is the desired dimension.

Strategy 2: Variance Threshold

Require that the latent space retains a proportion τ of the total variance:

$$\frac{\sum_{d=1}^{K} \lambda_d}{\sum_{d=1}^{D} \lambda_d} \geq \tau$$

Example: $\tau = 0.95$ means retaining 95% of the variance.

Strategy 3: Reconstruction Error

Train K such that $L(X) \leq \epsilon$, where for example:

$$L(X) = \sum_i \|x_i - \tilde{x}_i\|^2$$

Practical Advice: Visualizing the eigenvalue spectrum (scree plot) helps identify a natural "elbow".

Linear Autoencoders

Structure of a Linear Autoencoder

A linear autoencoder has:

- W_{enc} and W_{dec}: weight matrices (encoder and decoder)
- $z(x_i) = W_{\text{enc}} x_i$: encoding
- $\tilde{x}_i = W_{\text{dec}} z(x_i)$: decoding

Optimization Problem:

$$\theta^* = \underset{\text{rank}(W)=K}{\arg\min} \|X - WZ\|_F^2$$

> **Mathematical Details: Relationship Between PCA and Autoencoder**
>
> Using the Eckart-Young theorem with SVD decomposition $X = U\Psi V^T$, the solution is:
>
> $$W_{\text{dec}} Z = U_K \Psi_K V_K^T$$
>
> Setting $W_{\text{dec}} = U_K \Psi_K$, then:
>
> $$W_{\text{enc}} = \Psi_K^{-1} U_K^T$$
>
> For **centered** data:

- **PCA:** $\Sigma = \frac{1}{N} X X^T = U \Lambda U^T$ thus $\tilde{x}_i = U_K U_K^T x_i$
- **AE:** $X = U \Psi V^T$ thus $\Sigma = \frac{1}{N} U \Psi^2 U^T$ and $\tilde{x}_i = U_K U_K^T x_i$

Relationship between eigenvalues:

$$\Lambda = \frac{1}{N} \Psi^2$$

Fundamental Conclusion: A linear autoencoder performs PCA if the data is **centered and normalized**.

Note: W_{enc} is **not the inverse** of W_{dec} if $K < D$.

Extensions of PCA

Probabilistic PCA

Reformulation with explicit latent distribution:

- Latent distribution: $\mathbb{P}(z) = \mathcal{N}(0, I_{dK})$
- Conditional model: $\mathbb{P}(x|z) = \mathcal{N}(Wz + \mu, \sigma^2 I_{dD})$

Advantages:

- Accommodation of **measurement noise** (variance σ^2)
- **Generative mode**: possibility to generate new data
- Estimation by **Maximum Likelihood**
- **EM algorithm** to save computations

Kernel PCA

Use of a **nonlinear mapping** $\phi(x_i)$ via a kernel function:

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Allows performing PCA in a **more favorable space** (possibly infinite-dimensional) without explicitly computing ϕ.

Local PCA

Performs PCA on **local neighborhoods** of the data to consider only the **local intrinsic dimensionality**.

Useful for data with complex local structure (nonlinear manifolds).

What PCA Tells You (and What It Doesn't)

What It Tells You

- The **linear structure** of the data
- The **principal directions** of variation
- The **intrinsic dimension** (how many dimensions are truly useful)
- Which **variables** contribute to each axis

What It Doesn't Tell You

- **Not clustering**: It doesn't find groups
- **Linear only**: It misses curved relationships
- **Sensitive to outliers**: An extreme point can shift everything

---

When to Use PCA?

Suitable For

- Data approximately following a Gaussian distribution
- Dimensionality reduction with variance preservation
- Visualization of high-dimensional data
- Denoising data corrupted by Gaussian noise
- Data compression

Not Suitable For

- Strongly clustered data (non-Gaussian)
- Data with strongly nonlinear relationships
- Cases where variance is not a good information criterion

Condensed Cheat Sheet

In 30 seconds:

- PCA = smart basis change
- New axes = directions of maximum variance
- Eigenvalues = importance of each axis
- Always center first!
- Choose K with the scree plot
- Useful for: visualization, dimensionality reduction, denoising
- Not useful for: clustering, nonlinear relationships

Key Mathematical Concepts to Master

To understand PCA well:

- **Linear transformation** and **orthogonal matrix**
- **Coordinates** and **matrix rank**
- **Mean** and **variance**
- **Orthogonal projection**
- **Eigendecomposition**
- **Matrix trace**
- **Frobenius norm**
- **Lagrange multipliers**